

김민석

Frontend Engineer Portfolio

주식회사 센디 · 프론트엔드 엔지니어 | 2023.12.26 - 현재

min49590@gmail.com | 010-4601-4954 | [GitHub](#) | [LinkedIn](#) | [기술 블로그](#)

현재 주식회사 센디에서 근무하며, 복잡한 업무 흐름을 사용자가 이해하기 쉬운 웹 경험으로 구조화하고 변경 사항이 안전하게 배포·운영되도록 만드는 프론트엔드 엔지니어입니다.

Next.js, React, TypeScript 를 중심으로 제품 기능뿐 아니라 API·세션·상태·캐시, 테스트, CI/CD, 보안, 관측성, SEO·GEO 콘텐츠 시스템을 함께 다뤄 왔습니다. 특히 페이지 성능 개선을 계기로 센디 웹의 테크니컬 SEO 와 온서프 SEO 를 약 7 개월간 주도했습니다. Google Search Console 을 중심으로 지표를 추적하며 개선을 반복한 결과, 웹 트래픽이 약 3 배로 증가했습니다. 레거시를 한 번에 교체하기보다 호환성 경계와 롤백 가능성을 확보한 뒤 단계적으로 개선하며, 검증 결과와 남은 위험을 문서로 명확하게 남기는 방식을 선호합니다.

대표 성과

약 3 배

7 개월간 SEO 개선 후 웹 트래픽

약 199 개

App Router Route Handler 이전 API

최대 368 / 22

단위 테스트 / E2E 실행 증거

주요 역량

- 복잡한 운영 업무를 데이터 중심의 웹 UI 로 설계하고 구현합니다.
- 테스트와 검증을 코드 변경과 동일한 수준의 산출물로 관리합니다.
- 검색 성과를 정적 렌더링, 성능, 크롤링, 인덱싱, 구조화 데이터, 콘텐츠 운영의 통합 문제로 다루고 지표 기반으로 반복 개선합니다.
- 제품을 중단하지 않고 레거시 구조를 점진적으로 현대화합니다.
- 반복되는 개발 절차를 저장소 규칙, 스크립트, CI 자동화로 전환합니다.
- 운영 오류와 보안 위험을 실패 경계별로 분석하고 복구 가능한 방식으로 처리합니다.

대표 프로젝트

- 01 센디 웹 SEO 성장 및 테크니컬 SEO 운영 체계 구축
- 02 운송관리 핵심 화면을 그리드 기반 인라인 편집 UX 로 재설계
- 03 다중 프론트엔드 저장소의 AI QA·브라우저 인증·PR 검증 표준화
- 04 미사용 Jest 환경을 Vitest·MSW·Playwright 테스트 체계로 전환
- 05 검색엔진과 생성형 AI 가 이해할 수 있는 기업 뉴스룸 구축
- 06 운영 콘솔의 PWA 전환과 모바일 설치 가이드 구축

센디 웹 SEO 성장 및 테크니컬 SEO 운영 체계 구축

프로젝트 개요

CTO의 페이지 속도 개선 요청에서 출발해, 랜딩 페이지의 렌더링·성능 문제를 검색 노출과 사용자 유입의 문제로 확장했습니다. 공식적인 SEO 직책을 부여받아 시작한 업무는 아니었지만 실무 오너십을 자발적으로 맡았으며, 테크니컬 SEO와 온서프 SEO를 학습하고 코드에 적용한 뒤 Google Search Console 지표를 지속적으로 점검했습니다.

단발성 최적화로 끝내지 않고 정적 렌더링, Core Web Vitals, 시맨틱 HTML, 구조화 데이터, canonical, 사이트맵, 콘텐츠 시스템을 하나의 운영 체계로 연결했습니다. 약 7개월간 개선을 이어가는 동안 웹 트래픽이 약 3배로 증가하는 흐름을 확인했습니다.

해결해야 했던 문제

- A/B 테스트를 위해 서버 컴포넌트에서 요청별 쿠키와 헤더를 읽으면서 랜딩 페이지가 정적으로 생성되지 못했습니다.
- 폰트, CSS, JavaScript 번들, 비핵심 이미지가 함께 병목을 만들며 TTFB와 LCP를 악화시키고 있었습니다.
- 레거시 랜딩은 `div`와 `span` 중심으로 구성된 콘텐츠 계층이 검색엔진에 명확하게 전달되지 않았습니다.
- 유사 URL과 파라미터 URL의 대표 주소, 콘텐츠 섹션별 크롤링 경로, 구조화 데이터가 체계적으로 관리되지 않았습니다.
- SEO를 배포 전 체크리스트로만 다루지 않고, 검색 지표를 관찰하며 반복적으로 개선할 담당 방식이 필요했습니다.

주요 기여

- Lighthouse와 Chrome DevTools의 Performance·Network를 활용해 렌더링, 번들, 폰트, 이미지 로딩 병목을 구분했습니다.
- 랜딩 페이지를 `force-static`으로 고정하고 A/B 테스트 로직은 hydration 이후 클라이언트에서 적용하도록 분리했습니다.
- 실험 영역을 비핵심 카피·배지·버튼 문구로 제한하고 고정 크기 레이아웃을 적용해 CLS 위험을 통제했습니다.
- 사용하지 않는 의존성을 제거하고 무거운 컴포넌트와 서드파티 스크립트를 지연 로딩했습니다. 폰트 웨이트·서브셋, 불필요한 CSS, 이미지 `priority`와 `lazy loading`을 재정비했습니다.
- 레거시 랜딩을 `header`, `nav`, `main`, `section`, `footer` 등 시맨틱 HTML 기반으로 개선했습니다.
- `title`, `description`, Open Graph, canonical을 정비하고 페이지 성격에 맞춰 `Organization`, `FAQPage`, `HowTo`, `Article`, `BreadcrumbList` JSON-LD를 적용했습니다.
- Google의 리치 결과 테스트와 Search Console에서 구조화 데이터 인식과 오류 여부를 확인했으며, 실제 검색 결과에서 리치 스니펫이 표시되는 것을 확인했습니다.
- `/blog`, `/guide`, `/review` 등 섹션별 사이트맵과 sitemap index를 구성하고, 변경된 영역을 독립적으로 제출·점검할 수 있도록 했습니다.
- Google Search Console에서 노출수, 클릭수, CTR, 평균 게재 순위, 색인 상태, Core Web Vitals를 반복적으로 확인했습니다.
- 이후 뉴스룸, 공지사항, 업데이트 노트 등 검색엔진과 생성형 AI가 참조할 수 있는 정적 콘텐츠 기반으로 개선 범위를 확장했습니다.

핵심 설계 판단

- 1 정적 랜딩과 동적 실험의 경계를 분리했습니다. 검색엔진과 첫 방문자에게 전달되는 핵심 메시지는 정적 HTML에 유지하고, 요청별로 달라지는 실험 UI만 클라이언트에 격리했습니다.
- 2 성능 최적화를 측정 가능한 반복 과정으로 다뤘습니다. 추측으로 코드를 제거하지 않고 측정 → 제거 또는 지연 → 재측정 순서로 병목을 좁혔습니다.

- 3 SEO를 메타태그 작업으로 축소하지 않았습니다. 렌더링, 콘텐츠 계층, 검색 결과 표현, URL 정규화, 크롤링 경로, 콘텐츠 운영을 하나의 시스템으로 설계했습니다.
- 4 성과를 단일 변경에 귀속하지 않았습니다. 검색 트래픽은 여러 요인의 영향을 받으므로, 개별 구현이 3배 성장을 단독으로 만들었다고 표현하기보다 약 7개월간의 복합 개선과 지표 변화를 연결했습니다.

결과

- 2025년 10월부터 테크니컬 SEO와 온서프 SEO 개선을 반복했으며, 2026년 2월 시점에는 DAU와 CTR이 각각 약 2배로 상승하는 흐름을 확인했습니다.
- 이후 2026년 6월 회고에서는 개선을 시작한 지 약 7개월 만에 웹 트래픽이 약 3배로 증가했다고 기록했습니다.
- 검색 노출수, CTR, 평균 게재 순위가 개선됐으며, 구조화 데이터가 검색엔진에 인식되고 리치 스니펫이 실제 검색 결과에 표시됐습니다.
- SEO를 특정 개발 티켓이 아니라 지표 모니터링, 코드 개선, 콘텐츠 발행 기반이 순환하는 지속적인 업무로 정착시켰습니다.

사용 기술

Next.js App Router, React, TypeScript, static rendering, A/B experimentation, Lighthouse, Chrome DevTools, Core Web Vitals, Google Search Console, semantic HTML, JSON-LD, sitemap index, canonical, Open Graph

관련 글

- 샌디 SEO를 3배 성장시킨 이야기(1) - 어쩌다 SEO를 맡게 되었나
- Sendy에서 웹 테크니컬 SEO 최적화하기
- 프론트엔드 개발자가 알아야 할 SEO

PROJECT 02

운송관리 핵심 화면을 그리드 기반 인라인 편집 UX로 재설계

프로젝트 개요

운영자가 주문을 조회한 뒤 별도 화면을 오가며 수정하던 기존 흐름을, 엑셀과 유사하게 탐색·선택·수정할 수 있는 데이터 그리드 중심 경험으로 재설계했습니다. 기존 비즈니스 규칙과 API 계약을 유지하면서도 신규 화면을 안전하게 검증하고 전환할 수 있도록 구성했습니다.

해결해야 했던 문제

- 주문 정보를 확인하고 수정하기 위해 여러 화면과 폼을 반복해서 이동해야 했습니다.
- 주소, 운송일시, 차량, 고객 정보, 운임, 화물, 메모 등 필드별 편집 규칙이 달랐습니다.
- 신규 화면을 도입하면서도 기존 업무 흐름과 API 호환성을 유지해야 했습니다.
- 저장 이후 목록 전체를 다시 불러오면 사용성이 떨어지고 불필요한 네트워크 비용이 발생할 수 있었습니다.

주요 기여

- 기존 화면은 legacy 경로로 보존하고 신규 화면을 별도 경로에 구축해 비교와 롤백이 가능한 전환 구조를 만들었습니다.
- 필터 바, 데이터 그리드, 컬럼 크기·순서 저장, 다중 선택, 드래그 선택, 페이지 상태를 구현했습니다.

- 주소, 운송일시, 차량, 고객 정보, 운임, 화물, 메모 등 도메인별 셀 편집기를 분리해 설계했습니다.
- 단일 셀을 수정할 때 상세 응답으로 기존 폼 상태를 복원하고, 변경된 필드만 반영한 안전한 업데이트 payload 를 생성했습니다.
- 저장 후 목록 전체를 다시 조회하지 않고 React Query 캐시를 패치했습니다.
- 기존 운송 상세, 기사 배차, ETA, 운송 완료·취소 기능을 신규 그리드와 연결했습니다.
- 출시 전 기능은 UI 진입점을 비활성화해 구현 완료 시점과 실제 릴리스 시점을 분리했습니다.

핵심 설계 판단

- 1 전면 교체보다 점진적 전환을 선택했습니다. 신규 화면을 기존 화면과 분리해 운영 중인 기능을 보호하고, 문제가 발생하면 즉시 되돌릴 수 있도록 했습니다.
- 2 편집기를 도메인별로 분리했습니다. 모든 셀을 하나의 범용 컴포넌트로 처리하기보다 각 필드의 검증과 상호작용을 독립적으로 관리했습니다.
- 3 API 계약을 프론트엔드에서 안전하게 복원했습니다. 부분 편집 UI 를 제공하면서도 기존 API 가 요구하는 전체 payload 형식을 유지했습니다.
- 4 전체 재조회 대신 캐시를 갱신했습니다. 저장 직후 사용자에게 변경 결과를 빠르게 보여주면서 불필요한 데이터 요청을 줄였습니다.

결과

- 핵심 기능이 병합됐으며, 구현 계획과 후속 작업 상태가 저장소 문서에 함께 기록됐습니다.
- 신규 화면과 legacy 화면의 차이, API 저장 경계, 후속 QA 항목이 PR 리뷰 포인트로 정리됐습니다.
- UI, client state, API payload, 캐시, 상세 기능을 아우르는 end-to-end 프론트엔드 구현 경험을 확보했습니다.

사용 기술

Next.js, React, TypeScript, TanStack Query, Zustand, 데이터 그리드, route-local component·hook·store 구조

PROJECT 03

다중 프론트엔드 저장소의 AI QA·브라우저 인증·PR 검증 표준화

프로젝트 개요

Claude Code 와 Codex 같은 AI 에이전트가 실제 개발 서버를 안전하게 검증하고, 실행한 검사와 실행하지 않은 검사를 PR 에 정확하게 남길 수 있도록 공통 작업 계약을 설계했습니다.

해결해야 했던 문제

- 로그인된 브라우저 세션을 만들기 위해 테스트 계정이나 환경 변수를 에이전트에 노출할 위험이 있었습니다.
- 저장소마다 QA 방식과 PR 검증 기록 형식이 달랐습니다.
- 실제 개발 서버 QA 와 MSW 기반 E2E 가 혼동될 수 있었습니다.
- 에이전트가 실행하지 않은 검사를 통과한 것처럼 기록할 가능성이 있었습니다.
- PR 본문 자동화가 사람이 작성한 화면 설명과 리뷰 메모를 덮어쓸 수 있었습니다.

주요 기여

- Claude Code 와 Codex 에 공통으로 적용할 `authenticate-browser`, `qa-verify`, `pr-ready` 작업 계약을 설계했습니다.
- 자격증명은 전용 로컬 프로세스만 읽고, 에이전트는 권한이 제한된 브라우저 storage state 만 사용하도록 경계를 분리했습니다.
- `.env`, 테스트 계정 값, 저장된 쿠키 내용을 에이전트가 직접 읽지 못하도록 금지 규칙과 통합 테스트를 추가했습니다.
- 실제 개발 서버와 QA 계정을 사용하는 검증, MSW 기반의 반복 가능한 자동 E2E 를 명시적으로 구분했습니다.
- 명령, 종료 코드, 테스트 수, 실행 환경, 기준 커밋 SHA 를 evidence pack 으로 저장했습니다.
- PR 템플릿에서 자동 관리 영역과 사람이 작성하는 화면 설명·리뷰 메모·최종 확인 영역을 marker 로 분리했습니다.
- 테스트 실패는 차단하되, PR 댓글이나 artifact 게시 실패가 실제 테스트 결과를 덮어쓰지 않도록 CI 를 구성했습니다.

핵심 설계 판단

- 1 **비밀 정보와 실행 권한을 분리했습니다.** 에이전트가 인증 결과는 사용할 수 있지만 인증 정보 자체는 읽을 수 없도록 구성했습니다.
- 2 **검증 종류를 구분했습니다.** 실제 서버의 통합 상태를 확인하는 QA 와 외부 의존성을 차단한 반복 가능한 E2E 를 서로 다른 증거로 관리했습니다.
- 3 **검증 결과를 커밋과 연결했습니다.** 어떤 코드 상태에서 어떤 명령을 실행했는지 추적할 수 있도록 기준 SHA 와 실행 환경을 함께 기록했습니다.
- 4 **자동화와 사람의 판단 영역을 분리했습니다.** 자동 생성 정보가 리뷰어의 판단과 최종 승인 내용을 덮어쓰지 않도록 했습니다.

결과

- 한 저장소에서 단위 테스트 368 건, E2E 18 건, build 가 통과했습니다.
- 다른 저장소에서 단위 테스트 354 건, E2E 22 건이 통과했으며, flaky 결과 1 건을 숨기지 않고 기록했습니다.
- 기사 웹에서는 단위 테스트 33 건과 E2E 15 건이 통과했고, 자동 E2E 에서 외부 HTTP 요청이 차단되는 것을 확인했습니다.
- 기존 lint 기준선 오류와 신규 변경으로 발생한 오류를 base/head 비교로 분리했습니다.
- 도입 당시 일부 저장소의 관련 PR 이 open 또는 draft 상태였으므로, 모든 저장소에 최종 정착됐다고 표현하지 않았습니다.

사용 기술

Playwright, MSW, Vitest, GitHub Actions, browser storage state, Shell·Node.js 자동화, Markdown 기반 agent skill

PROJECT 04

미사용 Jest 환경을 Vitest·MSW·Playwright 테스트 체계로 전환

프로젝트 개요

설정과 의존성만 남아 있고 실제로 활용되지 않던 Jest 환경을 제거하고, Next.js·TypeScript 코드베이스에서 실행 가능한 단위 테스트 와 E2E, CI 검증 체계를 구축했습니다.

해결해야 했던 문제

- 테스트 의존성과 설정은 있었지만 실제 테스트가 거의 실행되지 않았습니다.
- TypeScript path alias 가 많아 테스트 환경에서도 실제 애플리케이션과 동일하게 모듈을 해석해야 했습니다.
- 외부 API 의존성 때문에 테스트 재현성이 떨어질 수 있었습니다.

- 초기 도입 단계에서 모든 E2E 를 곧바로 CI 필수 게이트로 적용하면 운영 부담이 커질 수 있었습니다.

주요 기여

- Jest 와 관련 설정을 제거하고 Vitest, jsdom, Testing Library, jest-dom 을 도입했습니다.
- MSW Node server 와 공통 lifecycle setup 을 추가해 외부 API 의존성을 격리했습니다.
- 기존 테스트의 Jest API 와 mock 을 Vitest 방식으로 이관했습니다.
- vite-tsconfig-paths 를 적용해 TypeScript alias 를 테스트에서도 동일하게 해석했습니다.
- Playwright 설정과 랜딩 페이지 smoke test 를 추가했습니다.
- GitHub Actions 에서 type generation, build metadata 생성, typecheck, unit test 가 순서대로 실행되도록 구성했습니다.
- 로컬 실행 명령, 파일 위치, CI 와 로컬 E2E 의 범위를 문서화했습니다.

핵심 설계 판단

- 도구만 교체하지 않고 기존 테스트, alias, mock, CI, 실행 문서를 함께 이관했습니다.
- 초기 E2E 는 로컬 smoke 검증으로 제한하고, 안정성을 확보한 뒤 적용 범위를 넓히는 방식을 선택했습니다.

결과

- 기존 테스트가 새 러너에서 실행 가능한 상태로 전환됐습니다.
- 단위 테스트, API mock, 브라우저 smoke test, CI 검증이 하나의 테스트 운영 체계로 연결됐습니다.
- 이후 동일 계열 저장소에서 Vitest, MSW, Playwright 가 공통 품질 도구로 확장됐습니다.

사용 기술

Vitest, Testing Library, jsdom, MSW, Playwright, GitHub Actions, TypeScript path alias

PROJECT 05

검색엔진과 생성형 AI 가 이해할 수 있는 기업 뉴스룸 구축

프로젝트 개요

테크니컬 SEO 로 확보한 검색 성장 기반을 콘텐츠 SEO 와 GEO 로 확장하기 위해, 여러 출처에 흩어진 기업 보도자료와 외부 언론 보도를 자사 도메인의 일관된 근거 페이지로 정리했습니다. 검색엔진과 생성형 AI 가 기업의 활동·파트너십·성적을 구조적으로 이해할 수 있도록 뉴스룸을 구축했습니다.

해결해야 했던 문제

- 기업 활동, 파트너십, 성과 관련 정보가 여러 외부 출처에 흩어져 있었습니다.
- 기존 블로그와 구분되는 콘텐츠 모델과 운영 규칙이 필요했습니다.
- 외부 기사를 그대로 복제하지 않으면서 사실관계를 유지해야 했습니다.
- 페이지 구현뿐 아니라 metadata, canonical, sitemap, 구조화 데이터까지 일관되게 연결해야 했습니다.

주요 기여

- /newsroom 목록 페이지와 정적 상세 페이지를 구축했습니다.

- 기존 블로그와 저장 형식은 공유하되 뉴스룸의 타입, parser, cache, query, component 를 별도 도메인으로 분리했습니다.
- 사건 날짜 기준 정렬, 이전·다음 글, 목록 탐색을 구현했습니다.
- canonical, 메타데이터, 공통 Open Graph 이미지, sitemap section 을 연결했습니다.
- 목록 페이지에는 `CollectionPage·ItemList`, 상세 페이지에는 `NewsArticle·BreadcrumbList` JSON-LD 를 적용했습니다.
- 외부 기사를 복제하지 않고 기업 관점으로 재작성하기 위한 콘텐츠 원칙과 사실 검수 기준을 문서화했습니다.
- 초기 과거 자료를 Markdown 콘텐츠로 이관하고 출처 검증 정보를 보강했습니다.

핵심 설계 판단

- 1 뉴스룸을 독립된 콘텐츠 도메인으로 분리했습니다. 기존 블로그 인프라를 재사용하면서도 뉴스룸 고유의 타입과 탐색 규칙을 유지했습니다.
- 2 SEO·GEO 를 페이지 메타태그에 한정하지 않았습니다. 콘텐츠 작성, 출처 검수, 구조화 데이터, 사이트맵을 하나의 운영 체계로 연결했습니다.
- 3 외부 기사 복제 대신 재작성 원칙을 적용했습니다. 자사 관점의 맥락을 제공하면서도 출처와 사실관계를 검증할 수 있도록 했습니다.

결과

- 뉴스룸 기능과 초기 콘텐츠가 병합됐습니다.
- 검색·GEO 요구를 페이지 구현, 콘텐츠 운영 규칙, 구조화 데이터, 사이트맵까지 하나의 시스템으로 연결했습니다.
- 전체 SEO 개선 기간의 웹 트래픽 성장은 프로젝트 1 에 기재했지만, 뉴스룸 단독 기여도와 생성형 AI 인용 증가율은 분리 측정하지 않아 별도 성과로 주장하지 않았습니다.
- PR 작성 시점에는 일부 단위 테스트와 목록·상세 수동 QA 가 재실행되지 않았다는 점을 명시했습니다.

사용 기술

Next.js static routes, Markdown, content parser·cache, sitemap, canonical, JSON-LD, Open Graph

PROJECT 06

운영 콘솔의 PWA 전환과 모바일 설치 가이드 구축

프로젝트 개요

모바일에서 운영 콘솔을 반복 사용하는 사용자가 브라우저 주소를 매번 입력하지 않고 홈 화면에서 앱처럼 실행할 수 있도록 PWA 와 플랫폼별 설치 안내를 구현했습니다.

해결해야 했던 문제

- 모바일 사용자가 운영 콘솔에 반복적으로 접근하기 불편했습니다.
- production, staging, local 환경별로 앱 이름과 아이콘을 구분해야 했습니다.
- iOS 와 Android 의 설치 방식이 서로 달랐습니다.
- service worker, manifest, icon, offline route 가 인증 proxy 에 의해 차단될 수 있었습니다.

주요 기여

- Serwist 기반 manifest, service worker, offline fallback, build pipeline 을 추가했습니다.

- production, staging, local 환경별 PWA metadata, title, icon 을 분리했습니다.
- 모바일에서 미설치 상태를 감지해 설치 가이드 modal 을 노출했습니다.
- iOS 에서는 공유 메뉴의 홈 화면 추가 절차를 안내하고, Android 에서는 `beforeinstallprompt` 를 활용한 설치 버튼을 제공했습니다.
- service worker, manifest, icon, offline route 가 인증 proxy 를 통과하지 않도록 matcher 예외를 추가했습니다.

결과

- typecheck 가 통과했고 아키텍처 하네스에서 신규 위반이 발생하지 않았습니다.
- 기능은 병합됐습니다.
- 도입 당시 모바일 실기기 설치 흐름이 staging 또는 production 배포 후 확인 과제로 남아 있었습니다.

사용 기술

Next.js, React, Serwist, Web App Manifest, Service Worker, beforeinstallprompt, responsive modal UX

기타 주요 기술 기여

App Router 및 세션 구조 현대화

- 약 199 개의 API relay 를 App Router Route Handler 로 이전했습니다.
- 결제·인증 callback 은 기존 구조에 남겨 세션과 redirect 위험을 분리했습니다.
- iron-session 메이저 업그레이드와 App Router·Pages Router 공용 session entry point 를 구성했습니다.

코드베이스 계층화 및 타입 안정성 강화

- `app`, `core`, `shared`, `features`, `api` 계층과 기능별 모듈을 도입했습니다.
- TypeScript strict 전환과 Biome 규칙 강화로 타입 안정성 기준을 높였습니다.
- dependency-cruiser 와 knip 을 활용해 구조 위반과 dead code 를 점검했습니다.

Sentry 기반 운영 오류 개선

- hydration, 외부 SDK 준비 시점, API 실패, WebView bridge, 잘못된 URL 상태를 원인별로 분리했습니다.
- 오류를 일괄적으로 숨기기보다 준비 상태 대기, null guard, 404 전환, 재로그인 안내 등 실패 경계에 맞는 대응을 적용했습니다.

보안 및 배포 안정성 개선

- `npm audit` prebuild gate, 런타임·의존성 패치, 릴리스 경과일 제한, lifecycle script 차단, 전이 의존성 override 를 적용했습니다.
- Docker 와 ECS 배포 workflow 를 정비하고 rollback 검증, 동시 배포 방지, BuildKit secret, health check, 실패 원인 알림을 구성했습니다.

업무 방식

- 문제를 UI, client state, API, session, 외부 SDK, CI 와 같은 경계 단위로 나눠 원인을 좁힙니다.
- 기존 계약과 롤백 경로를 먼저 확보한 뒤 새 구조를 도입합니다.

- 변경 범위에 맞춰 typecheck, unit test, E2E, browser QA, build, architecture check 를 선택합니다.
- 기존 기준선 오류와 신규 회귀를 구분해 보고합니다.
- 실행하지 않은 테스트, flaky 결과, 실기기 미검증 항목을 숨기지 않고 기록합니다.
- 반복되는 작업은 문서, 스크립트, CI, agent rule 로 남겨 다른 개발자나 AI 에이전트가 이어갈 수 있도록 합니다.

기술 스택

- **Frontend:** Next.js 16, React 19, TypeScript 5, App Router, TanStack Query, Zustand, Tailwind CSS, Storybook
- **Quality:** Biome, TypeScript strict, dependency-cruiser, knip, React Doctor, Husky
- **Platform:** GitHub Actions, Docker, ECS, Nginx, PWA, Service Worker
- **AI-assisted Engineering:** Claude Code, Codex, agent guardrails, browser QA session isolation, PR evidence automation
- **Testing:** Vitest, Testing Library, MSW, Playwright
- **Observability & Experimentation:** Sentry, Datadog, Mixpanel, A/B experimentation SDK
- **Content & Search:** Google Search Console, Core Web Vitals, SSG·ISR, sitemap index, canonical, Schema.org JSON-LD, Markdown content pipeline, Open Graph